

SkillClaw: Let Skills Evolve Collectively with Agentic Evolver

Collective Skill Evolution for Multi-User LLM Agent Ecosystems

arxiv.org/abs/2604.08377



Agent Skills Remain Static After Deployment — Users Rediscover Solutions Independently

STATIC SKILL PARADIGM

- Skills are manually installed from a centralized hub and never updated post-deployment
- Memory-based methods store past trajectories but remain tied to specific instances
- Skill-based methods compress experience into instructions but treat the library as frozen
- Each user is forced to rediscover solutions independently — no system-level accumulation

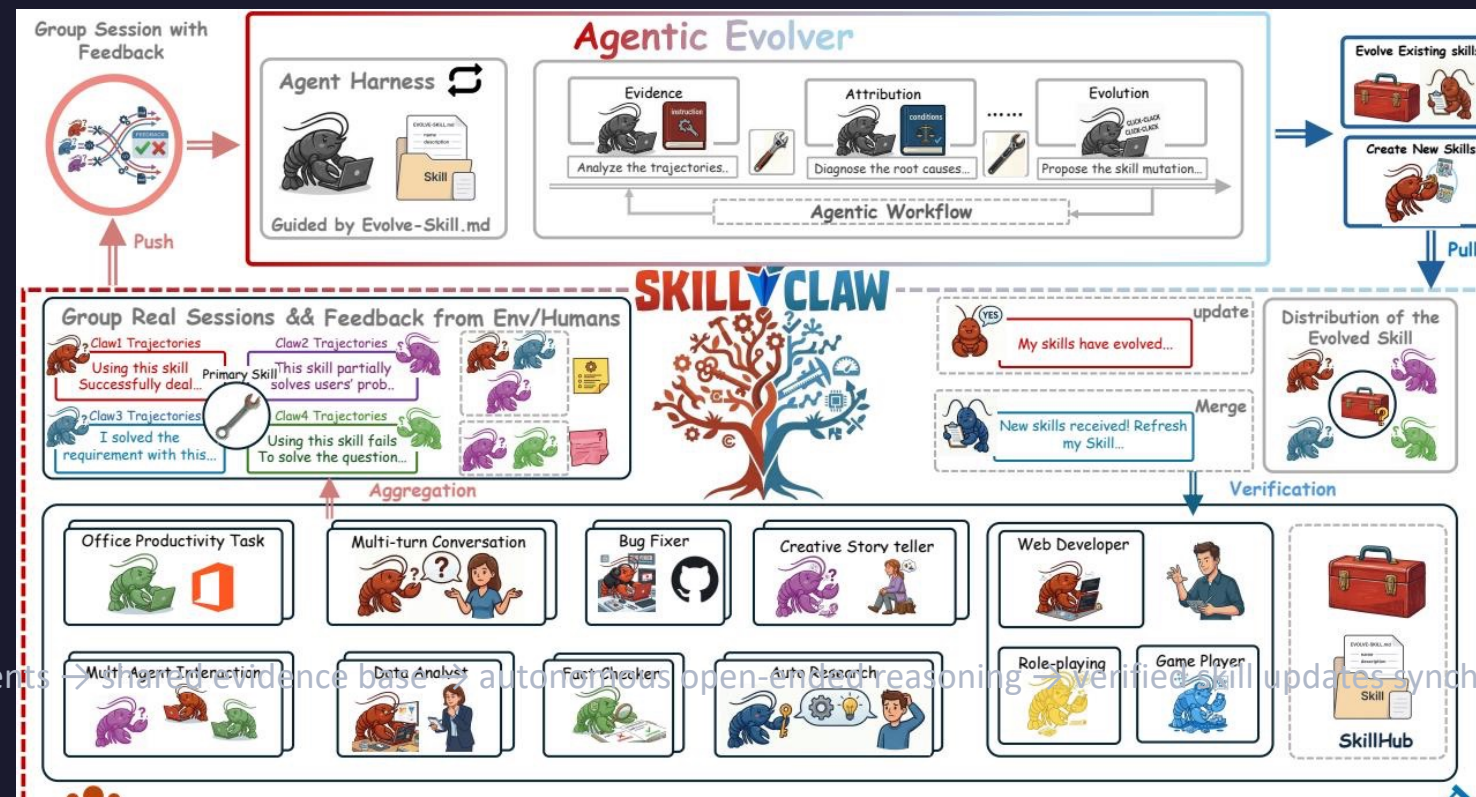
EVOLVING SKILL PARADIGM (SkillClaw)

- Runtime interaction trajectories are preserved with full action–feedback causal chains
- Cross-user evidence is aggregated into a shared, continuously improving skill ecosystem
- An agentic evolver performs open-ended reasoning to propose skill mutations
- Verified skills are synchronized back to all agents — knowledge accumulates automatically

Closed-Loop Pipeline: From Multi-User Interaction to Shared Skill Evolution



Four Stages Inside the Agentic Evolver: Evidence → Attribution → Evolution → Distribution



Independent agents → Share Evidence base → Automate open-ended reasoning → Verified skill updates synchronized to all agents

Agentic Evolver: Evidence → Attribution → Evolution in Three Stages

1. EVIDENCE

Analyze trajectories to identify behavioral patterns

- Parse structured session trajectories preserving action–feedback causal chains
- Group trajectories by referenced skills to form a skill-specific evidence base
- Surface success patterns, failure modes, and boundary conditions

2. ATTRIBUTION

Diagnose root causes of failures and successes

- Cross-reference failures against successful executions under similar contexts
- Distinguish generalizable issues from idiosyncratic user-specific fixes
- Identify whether a failure stems from skill deficiency or environment variance

3. EVOLUTION

Propose skill mutations through open-ended reasoning

- Refine existing skills or create new ones based on diagnosed root causes
- Driven by autonomous agent reasoning — not predefined update rules or templates
- Proposals are verified before merging to prevent regressions

Key distinction: the evolver is an autonomous agent performing open-ended reasoning over interaction evidence — not a rule-based updater.

Cross-User Trajectory Aggregation Exposes Skill Behavioral Boundaries

- Different users exercise the same skill under diverse contexts
- Each interaction produces complementary views of the skill's behavioral boundary
- Reveals both conditions where the skill works and where it breaks
- A single user rarely generates enough signal to separate a generalizable improvement from an idiosyncratic fix

COMPATIBLE CLAW-STYLE SYSTEMS

- OpenClaw
- IronClaw
- ZeroClaw
- NemoClaw
- CoPaw
- PicoClaw
- NanoClaw

Why aggregation matters

- Cross-user trajectories are grouped by referenced skills, forming a shared evidence base
- The diversity of user contexts provides the statistical signal needed for generalizable skill mutations
- Without it, the evolver cannot distinguish a one-off workaround from a robust improvement

Case Study: Multimodal Creative Pipeline Skill — 6 Days of Evolution Attempts

Day	Candidate Skill	Skill Function	Validator	Next-Day Action
1	validate-tmp-workspace-inputs	Check /tmp_workspace inputs & environment setup	Accept	Upgrade to new best pool
2	multimodal-input-validation-and-setup	Multimodal input validation & output env init	Reject	Keep current best pool
3	multimodal-creative-task-pipeline	Multimodal creative pipeline (extract from PDF/video/image)	Reject	Keep current best pool
4	multimodal-creative-task-pipeline (impr.)	Added image classification, visual generation, structured validation	Reject	Keep current best pool
5	multimodal-creative-task-pipeline (impr.) + validate-required-input-files	Creative pipeline + per-file fail-fast validation	Reject	Keep current best pool
6	multimodal-creative-task-pipeline (cand.)	Extended PDF-to-poster / document-to-visual paths	Reject	Not admitted to next cycle

Result: Only 1 of 6 candidates accepted — illustrating the conservative verification process that prevents regressions.

Case Study: Git Push Auth-Fallback Skill — Iterative Refinement Over 6 Days

Day	Candidate Skill	Skill Function	Validator	Next-Day Action
1	git-push-with-auth-fallback	Safe fallback instead of blocking on push failure	Accept	Add to Safety best pool
2	git-push-with-auth-fallback	Unified patch/bundle filenames, reduced inconsistency	Accept	Keep updated best pool
3	git-push-with-auth-fallback + git-clone-to-directory	Auth-alternative paths & secrets audit; fixed subshell pitfalls	Accept	Keep current best pool
4	(none)	Same-pool retest; validator confirmed current pool as best	Accept	Continue same best pool
5	git-push-with-auth-fallback	Added 'push hang treated as auth failure'; no improvement	Reject	Keep current best pool
6	git-push-with-auth-fallback	Added identity config & filename consistency; no improvement	Reject	Not admitted to next cycle

Result: 3 of 6 days yielded accepted updates — successful iterative refinement before reaching a plateau.

Key Takeaways: Three Properties That Distinguish SkillClaw from Existing Systems

01 COLLECTIVE EVOLUTION

Knowledge from individual interactions contributes to a shared, continuously improving skill ecosystem. Cross-user trajectory aggregation provides the complementary signal needed for generalizable mutations.

02 FULLY AUTOMATIC

Skill evolution is driven by runtime interaction without manual curation or explicit user intervention. The system improves while users simply work.

03 AGENTIC EVOLUTION PARADIGM

Skill updates are produced through open-ended reasoning by an autonomous evolver agent — not predefined update rules, templates, or heuristics.

Implication: SkillClaw points toward a future where agent capabilities are not fixed at deployment but continuously sharpened by the collective experience of their user communities.